CAPSTONE 31 — 10-Question Assessment Quiz & Answer Key

Diagnostic & Summative Evaluation of Branching, PRs, GitHub Actions CI & AI Ethics

Part A: 10-Question Diagnostic & Summative Assessment

Instructions for CSE Student Teams: Select the single best answer for each question. This assessment evaluates practical mastery of Git branching, Pull Requests, GitHub Actions CI workflows, local automation hooks, and Responsible AI engineering ethics.

Q1. [Branching Mechanics] In Git, what actually happens under the hood when you run `git switch -c feat/search-api`?
A) Git copies the entire project folder into a new physical directory on your hard drive.
B) Git creates a lightweight, movable 41-byte pointer (reference) to the current commit snapshot and updates `HEAD` to point to it.
C) Git locks the `main` branch on GitHub's servers so no other teammate can push.
D) Git automatically opens a Pull Request on GitHub.

Q2. [GitHub Flow Strategy] Why is **GitHub Flow** generally preferred over **Gitflow** for a 4-student 2nd- or 3rd-year CSE semester project?
A) GitHub Flow does not use branches at all.
B) GitHub Flow keeps a single deployable `main` branch with short-lived feature branches and PRs, avoiding the multi-branch merge overhead of maintaining separate `develop`, `release/*`, and `hotfix/*` branches.
C) Gitflow does not support automated unit testing.
D) GitHub Flow allows developers to force-push directly to `main` without code review.

Q3. [Fast-Forward vs. Three-Way Merge] You branched `feat/ui` off commit `C2` on `main`. While you made commits `C3` and `C4` on `feat/ui`, a teammate merged another PR that advanced `main` to `C5`. When `feat/ui` is merged into `main`, how does Git combine the histories?
A) Git performs a three-way merge using the common ancestor (`C2`) and the two branch tips (`C4` and `C5`), creating a new merge commit with two parents.
B) Git performs a fast-forward merge by moving the `main` pointer straight to `C4`.
C) Git deletes commit `C5` automatically.
D) Git refuses to merge divergent branches under any circumstances.

Q4. [Pull Request Templates] Where should you place a Markdown file in your repository so that GitHub automatically populates its checklist into the description box of every new Pull Request?
A) `.git/config/pr_template.txt`
B) `src/templates/pull_request.html`
C) `.github/pull_request_template.md`
D) `.github/workflows/pr_template.yml`

Q5. [Branch Protection Rules] A student team enables branch protection on `main` with *"Require a pull request before merging (1 approval)"* and *"Require status checks to pass before merging (flake8 &*

pytest)". What happens if a developer opens a PR where a unit test fails, even after a teammate clicks "Approve"?
A) GitHub merges the PR anyway because a human teammate approved it.
B) GitHub blocks the merge button until a new commit pushed to the PR branch causes the required CI status checks to pass.
C) GitHub automatically deletes the failing unit test.
D) GitHub reverts the repository to its initial commit.

Q6. [GitHub Actions Triggers & Matrix] In `.github/workflows/ci.yml`, what is the effect of configuring `on: [push, pull_request]` on `main` alongside `strategy: matrix: python-version: ["3.11", "3.12"]`?
A) It runs the job once using Python 3.11 on push and Python 3.12 on pull request.
B) It spawns two parallel isolated runner jobs (one on Python 3.11, one on Python 3.12) whenever code is pushed to `main` or a PR targeting `main` is opened/updated.
C) It upgrades the developer's local laptop Python version to 3.12.
D) It prevents the workflow from running on Ubuntu Linux.

Q7. [Static Analysis vs. Unit Testing] Why does our CI pipeline run **both** `flake8` and `pytest` in separate steps?
A) `flake8` performs static code analysis (catching syntax errors, undefined variable names, and PEP 8 style defects without executing the code), whereas `pytest` dynamically executes functions against test assertions to verify runtime logic.
B) `pytest` only checks Markdown formatting while `flake8` runs database migrations.
C) They do the exact same thing, so running both just doubles CI billing time.
D) `flake8` compiles Python into C++ binaries before `pytest` can run.

Q8. [Git Hook Automation] Which client-side Git hook receives the path to the temporary file containing the developer's commit message (`$1`) and can abort the commit by exiting with a non-zero status code if the message violates Conventional Commits?
A) `post-checkout`
B) `pre-push`
C) `commit-msg`
D) `post-merge`

Q9. [AI Hallucination & Outdated Advice Audit] You ask an AI chatbot to generate a GitHub Actions workflow, and it outputs `uses: actions/checkout@v2` and `run: echo "::set-output name=status::ok"`. Why must you reject or update this YAML before committing?
A) GitHub Actions no longer supports YAML files.
B) Both `actions/checkout@v2` (Node.js 12 runtime) and the `::set-output` workflow command were officially deprecated and disabled by GitHub due to security and runtime EOL; modern workflows use `actions/checkout@v4` (or `@v6`) and `$GITHUB_OUTPUT`.
C) The `echo` command is banned on Ubuntu runners.
D) `actions/checkout` only works for Java repositories.

Q10. [DevOps & AI Ethics] A teammate accidentally commits a `.env` file containing a live OpenAI/Gemini API key to a feature branch, pushes it to a public GitHub repo, and then deletes the file in a follow-up commit. What is the correct ethical and security response?
A) Do nothing; deleting the file in a new commit erases it from Git history.
B) Immediately revoke and rotate the exposed API key in the provider dashboard, remove the secret

from Git history, store the new key in GitHub Encrypted Secrets (`${{ secrets.API_KEY }}`), and ensure `.env` is listed in `.gitignore`.
C) Rename the branch so automated bots cannot find it.
D) Base64-encode the API key and commit it directly inside `.github/workflows/ci.yml`.

Part B: Answer Key & Detailed Technical Rationales

Q#	Correct Answer	Technical Rationale & Official Source Citation
Q1	**B**	Per Pro Git Ch. 3.1, a Git branch is simply a lightweight movable pointer to a commit object, making branching nearly instantaneous.
Q2	**B**	Per GitHub Flow Docs, a single protected `main` branch plus short-lived feature branches minimizes merge complexity for small agile/student teams.
Q3	**A**	Per Pro Git Ch. 3.2, when histories diverge from a common ancestor, Git performs a 3-way merge and creates a new merge commit.
Q4	**C**	Per GitHub PR Template Docs, placing `pull_request_template.md` in `.github/`, root, or `docs/` auto-populates new PR descriptions.
Q5	**B**	Per GitHub Protected Branch Docs, required status checks and required reviews are conjunctive (AND) gates; both must pass before merging.
Q6	**B**	Per GitHub Actions Python Guide, matrix strategies run separate jobs for each configured Python version across specified triggers.
Q7	**A**	Static linting (`flake8`) inspects AST/source formatting without running code, while unit testing (`pytest`) verifies functional correctness and edge cases at runtime.
Q8	**C**	Per Pro Git Ch. 8.3 (Git Hooks), the `commit-msg` hook validates and can reject a commit based on its message content before the commit is recorded.
Q9	**B**	LLMs trained on older web corpora frequently emit deprecated `@v2` actions and `::set-output` commands; always verify against current GitHub Actions Docs.
Q10	**B**	Per GitHub Actions Secure Use Reference, any secret committed to Git history must be considered compromised and rotated immediately, as prior commits remain readable.